

Enhancement in Out Variables in C# 7 with Examples

In this article, I am going to discuss the **Enhancement of Out Variables in C# 7** with Examples. With the Introduction of C# 7, now it is possible to define the method's out parameters directly within the method. As part of this article, we are going to discuss the following pointers.

1. [How to Work with the Out Parameter Before C# 7?](#)
2. [Working with Out Variable in C# 7.](#)
3. [What is the Scope of Out Parameter?](#)
4. [Can We Declare the Out Variable with Var Data Type From C# 7?](#)
5. [How to Ignore an Out Parameter in C#?](#)
6. [Out Parameter Using TryParse](#)

How to Work with the Out Parameter Before C# 7?

In C#, we generally use the **out parameter** to pass a method argument's reference. That means if you want to implement the call by reference in C#, then you need to use the OUT parameter. If you want to use the out parameter in C#, then you need to explicitly specify the **out keyword** in both the calling method as well as inside the method definition.

Before C# 7, we need to split their declaration and usage into two parts i.e. first we need to declare a variable and then we need to pass that variable to the method using the **out** keyword. The **Out** Parameter in C# never carries value into the method definition. So, it is not required to initialize the out parameter while declaring.

Example to Understand Out Parameter Before C# 7

Let us understand how to use Out Parameter Before C# 7 with an example. Please have a look at the below example. As you can see the **GetEmployeeDetails** method is created with four out parameters. Then within the Main method, first we declare four variables without initializing. Initialization is optional. Then while calling the GetEmployeeDetails method, we pass the four variables by specifying the out keyword.

```
using System;
class Program
{
    static void Main()
    {
        string EmployeeName, Gender, Department;
        long Salary;
        GetEmployeeDetails(out EmployeeName, out Gender, out Salary, out Department);
        Console.WriteLine("Employee Details:");
        Console.WriteLine("Name: {0}, Gender: {1}, Salary: {2}, Department: {3}",
            EmployeeName, Gender, Salary, Department);
        Console.WriteLine("Press any key to exit.");
        Console.ReadKey();
    }
    static void GetEmployeeDetails(out string EmployeeName, out string Gender, out
Salary, out string Department)
    {
        EmployeeName = "Pranaya Rout";
        Gender = "Male";
        Salary = 20000;
        Department = "IT";
    }
}
```

```
}  
}
```

Output:

```
Employee Details:  
Name: Pranaya Rout, Gender: Male, Salary: 20000, Department: IT  
Press any key to exit.
```

Note: The Out Parameter in C# never carries the value into the method definition. So, it is mandatory to initialize the out variables within the method definition otherwise you will get a compile-time error. Again you cannot use the “var” data type to declare these variables.

Now, the question is, if it is not required to initialize the out variables then why should we split their usage into two parts? Well, this will be overcome with C# 7.

Working with Out Variable in C# 7.

With the introduction of C# 7, now it is possible to define a method's out parameters directly while calling the method. So the above program can be rewritten as shown below and also give us the same output. Here, you can see, we are directly declaring the variable at the time of method call i.e. GetEmployeeDetails(out string EmployeeName, out string Gender, out long Salary, out string Department);. This will eliminate the need to split the usage of the C# out variable into two parts.

```
using System;  
class Program  
{  
    static void Main()  
    {  
        GetEmployeeDetails(out string EmployeeName, out string Gender, out long Salary, out string Department);  
        Console.WriteLine("Employee Details:");  
        Console.WriteLine("Name: {0}, Gender: {1}, Salary: {2}, Department: {3}",  
            EmployeeName, Gender, Salary, Department);  
        Console.WriteLine("Press any key to exit.");  
        Console.ReadKey();  
    }  
    static void GetEmployeeDetails(out string EmployeeName, out string Gender, out long Salary, out string Department)  
    {  
        EmployeeName = "Pranaya Rout";  
        Gender = "Male";  
        Salary = 20000;  
        Department = "IT";  
    }  
}
```

Run the application and you will get the output as expected as our previous program.

What is the Scope of Out Parameter in C#?

As we declared the out parameter directly within the method call, so we need to understand the scope of the out parameter in C#. In the above program, the out variables are in the scope of the enclosing block i.e. the enclosing block of the Main method in our example. So the subsequent line can use them.

Can we Declare the Out Variable with var Data Type?

From C# 7, yes, you can declare the out variables with the var data type. As the **out** variables are declared directly as arguments to the out parameters, so, the compiler can easily tell what their data type should be. So it is always better to use the “**var**” data type to declare them as shown in the following example.

```
using System;
class Program
{
    static void Main()
    {
        GetEmployeeDetails(out var EmployeeName, out var Gender, out var Salary, out
Department);
        Console.WriteLine("Employee Details:");
        Console.WriteLine("Name: {0}, Gender: {1}, Salary: {2}, Department: {3}",
EmployeeName, Gender, Salary, Department);
        Console.WriteLine("Press any key to exit.");
        Console.ReadKey();
    }
    static void GetEmployeeDetails(out string EmployeeName, out string Gender, out
Salary, out string Department)
    {
        EmployeeName = "Pranaya Rout";
        Gender = "Male";
        Salary = 20000;
        Department = "IT";
    }
}
```

How to Ignore an Out parameter in C#?

If you want to ignore an out parameter then you need to use a wildcard called **underscore** ('_') as the name of the parameter. For example, if you don't care about the **Department** parameter, then you just replace it with an **underscore** ('_') as shown below.

```
using System;
class Program
{
    static void Main()
    {
        GetEmployeeDetails(out var EmployeeName, out var Gender, out var Salary, ou
Console.WriteLine("Employee Details:");
        Console.WriteLine("Name: {0}, Gender: {1}, Salary: {2}",
EmployeeName, Gender, Salary);
        Console.WriteLine("Press any key to exit.");
        Console.ReadKey();
    }
    static void GetEmployeeDetails(out string EmployeeName, out string Gender, out
Salary, out string Department)
    {
```

```

{
    EmployeeName = "Pranaya Rout";
    Gender = "Male";
    Salary = 20000;
    Department = "IT";
}
}

```

OUTPUT:

```

Employee Details:
Name: Pranaya Rout, Gender: Male, Salary: 20000
Press any key to exit.

```

Out Parameter Using TryParse in C#

When we are working with Real-Time applications, then the common use of the **out** variable is the **Try... Pattern**, where a boolean return value indicates the success, and if successful then the **out** parameters carry the results. Let us understand this with an example.

Example to Understand Try Pattern using Out variable Before C# 7

Let us first see an example of using the C# out variable with a try pattern before C# 7. Please have a look at the following example. In the below example, first, we declare and initialize a string variable and then we declare a DateTime variable. Then within the if block we are calling the **DateTime.TryParse** and pass the first parameter as the string variable and the second one as the out DateTime parameter. If the above string is converted to DateTime, then **DateTime.TryParse** method will return true and the converted value will be stored in the out variable in C#.

```

using System;
class Program
{
    static void Main()
    {
        string s = "09-Jun-2018";
        DateTime date;
        if (DateTime.TryParse(s, out date))
        {
            Console.WriteLine(date);
        }
        Console.WriteLine("Press any key to exit.");
        Console.ReadKey();
    }
}

```

Output:

```

09-06-2018 00:00:00
Press any key to exit.

```

Example to Understand Try Pattern using Out Variable From C# 7

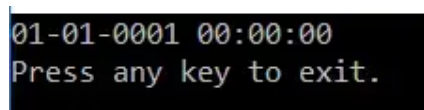
With the Introduction of C# 7, the previous example can be rewritten as shown below. As you can see, now we don't require to split the usage of the out variable into two parts. Directly we can declare the out variable within the method itself.

```
using System;
class Program
{
    static void Main()
    {
        string s = "09-Jun-2018";
        if (DateTime.TryParse(s, out DateTime date))
        {
            Console.WriteLine(date);
        }
        Console.WriteLine(date);
        Console.WriteLine("Press any key to exit.");
        Console.ReadKey();
    }
}
```

Run the application and it should give the same output as the previous example. In the above program, we are declaring the **out** variable within the method and it is being accessed from outside also. If an exception occurred, then the **out** variable will be assigned a default value. Let's see this with an example.

```
using System;
class Program
{
    static void Main()
    {
        string s = "09-Junnnneee-2018";
        if (DateTime.TryParse(s, out DateTime date))
        {
            Console.WriteLine(date);
        }
        Console.WriteLine(date);
        Console.WriteLine("Press any key to exit.");
        Console.ReadKey();
    }
}
```

Output:



```
01-01-0001 00:00:00
Press any key to exit.
```

In the next article, I am going to discuss one more interesting new feature of C# 7 i.e. [Pattern Matching in C# 7](#) with Example. Here, in this article, I try to explain the **Enhancement of the Out Variables in C# 7** with Examples. I hope you enjoy this Out variable in C# with Examples article.